

প্রফেশনাল প্রোগ্রামিং সিরিজ

পাইথন প্রোগ্রামিং মাস্টারক্লাস ও ওয়ার্কবুক

কোর ফাউন্ডেশন থেকে অ্যাডভান্সড আর্কিটেকচার ও রিয়েল
প্রজেক্টস

- ✓ সহজ বাংলা ব্যাখ্যা ও ভিজুয়াল মেকানিক্স
- ✓ ইন্ডাস্ট্রি স্ট্যান্ডার্ড কোড ডিজাইন প্যাটার্ন
- ✓ মেমোরি অপটিমাইজেশন ও প্রো-লেভেল কনসেপ্টস
- ✓ সম্পূর্ণ সোর্স কোড ও হ্যান্ডস-অন প্রজেক্টস

সূচিপত্র (Table of Contents)

🎯 **পার্ট ১: ফাউন্ডেশন বिल्ডিং (কোর পাইথন)**

অধ্যায় ১: প্রথম চিন্তাভাবনা ও সিনট্যাক্স	৪
অধ্যায় ২: ভ্যারিয়েবল ও ডেটা টাইপ	৭
অধ্যায় ৩: স্ট্রিং ও নাম্বার মেকানিক্স	১০
অধ্যায় ৪: পাইথনের পেশী (Lists & Loops)	১৩
অধ্যায় ৫: সিদ্ধান্ত গ্রহণ ও ডিকশনারি	১৬
অধ্যায় ৬: ইন্টারঅ্যাক্টিভিটি ও পুনঃব্যবহারযোগ্যতা	১৯

🧠 **পার্ট ২: প্রফেশনাল ও স্ট্রাকচার্ড পাইথন (OOP & Operations)**

অধ্যায় ১: অবজেক্ট-ওরিয়েন্টেড প্রোগ্রামিং (OOP)	২২
অধ্যায় ২: অ্যাডভান্সড OOP কনসেপ্ট	২৫
অধ্যায় ৩: পারসিস্টেন্ট স্টোরেজ ও এরর ডিফেন্স	২৮
অধ্যায় ৪: ইন্ডাস্ট্রি স্ট্যান্ডার্ড কোড টেস্টিং	৩১

🚀 **পার্ট ৩: অ্যাডভান্সড পাইথন ও মেমোরি মেকানিক্স**

অধ্যায় ১: মেমোরি ও মিউটেবিলিটি ডিপ ডাইভ	৩৪
অধ্যায় ২: অ্যাডভান্সড কমপ্রিহেনশন (Comprehensions)	৩৭
অধ্যায় ৩: পাইথনের স্পেশাল ফাংশন ও টুলস	৪০
অধ্যায় ৪: মেমোরি অপ্টিমাইজেশন (Iterators & Generators)	৪৩
অধ্যায় ৫: কোড মডিফিকেশন (Decorators)	৪৬
অধ্যায় ৬: ডান্ডার মেথডস (Dunder/Magic Methods)	৪৯
অধ্যায় ৭: রেগুলার এক্সপ্রেশন (Regex)	৫২

পার্ট ৪: প্রফেশনাল ওয়ার্কফ্লো, ইকোসিস্টেম ও রিয়েল প্রজেক্টস

অধ্যায় ১: এনভায়রনমেন্ট ম্যানেজমেন্ট	৫৫
অধ্যায় ২: প্রফেশনাল প্রজেক্ট আর্কিটেকচার	৫৭
অধ্যায় ৩: ক্লিন কোড ও প্রফেশনাল অভ্যাস	৫৮
অধ্যায় ৪: রিয়েল-ওয়ার্ল্ড প্রজেক্ট ইমপ্লিমেন্টেশন	৫৯

পার্ট ৫: দি আলটিমেট প্রো প্লাস (Pro+ Concepts)

অধ্যায় ১: কনকারেন্সি ও প্যারালেলিজম	৬৪
অধ্যায় ২: পাইথন ডেটা স্ট্রাকচার ডিপ ডাইভ	৬৭
অধ্যায় ৩: গিট ও গিটহাব প্রফেশনাল ওয়ার্কফ্লো	৬৯
অধ্যায় ৪: আপনার পরবর্তী রিয়েল-ওয়ার্ল্ড ডিরেকশন	৭০

পার্ট ১: ফাউন্ডেশন বিল্ডিং (কোর পাইথন)

অধ্যায় ১: প্রথম চিন্তাভাবনা ও সিনট্যাক্স (First Thoughts & Syntax)

১. সহজ বাংলা ব্যাখ্যা

- **লাইন বাই লাইন এক্সিকিউশন:** পাইথন হলো একটি **Interpreted** ভাষা। এর মানে হলো, পাইথন আপনার পুরো কোড একবারে না পড়ে, উপর থেকে নিচে (Top to Bottom) লাইনের পর লাইন ধরে পড়ে এবং সাথে সাথে কাজ সম্পন্ন করে। যদি ৩ নম্বর লাইনে কোনো ভুল থাকে, তবে পাইথন ১ ও ২ নম্বর লাইন ঠিকঠাক চালাবে এবং ৩ নম্বর লাইনে এসে চিৎকার করে বলবে—"আমি আর সামনে যাব না!"
- **কেস-সেন্সিটিভিটি (Case-Sensitivity):** পাইথনের চোখ খুব তীক্ষ্ণ। তার কাছে ছোট হাতের অক্ষর আর বড় হাতের অক্ষর এক নয়। যেমন: `apple` এবং `Apple` পাইথনের কাছে দুটি সম্পূর্ণ ভিন্ন জিনিস।
- **print() ফাংশনের মেকানিক্স:** `print()` হলো পাইথনের আউটপুট উইন্ডো। এর ভেতরে আপনি যা দেবেন, সে ক্ষিণে তা-ই ফুটিয়ে তুলবে। তবে মনে রাখবেন, লেখা বা টেক্সট দেখাতে চাইলে তাকে 'কোটেশন' (`'...'` বা `"..."`) এর ভেতরে আটকে দিতে হবে।
- **সিনট্যাক্স এরর (Syntax Error):** এটি হলো ব্যাকরণগত ভুল। পাইথনের নিজস্ব কিছু নিয়ম বা ব্যাকরণ আছে। আপনি যদি সেই নিয়ম ভাঙেন (যেমন: ব্র্যাকেট ক্লোজ না করা বা কোটেশন দিতে ভুলে যাওয়া), তবে পাইথন কোড রানই করবে না। একে বলে Syntax Error।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
# এটি একটি কमेंট। হাস (#) চিহ্ন দিলে পাইথন সেই লাইনটি পড়ে না।
print("Hello, Python World!") # দ্বৈত বা একক কোটেশন ব্যবহার করা যায়।

# লাইন বাই লাইন মেকানিক্স প্রদর্শন:
print("লাইন ১: এটি আগে রান হবে।")
print("লাইন ২: এটি পরে রান হবে।")
```

৩. সঠিক ও ভুল কোডের উদাহরণ

● CORRECT DESIGN PATTERN

```
print("Welcome to Python")  
print("Welcome to python") # দুটি আলাদা লাইন হিসেবে প্রিন্ট হবে, কোনো এরর আসবে না।
```

● INCORRECT DESIGN PATTERN

```
Print("Hello World") # ভুল! 'print'-এর P বড় হাতের দেওয়া যাবে না (Case-Sensitivity Error).  
print("Hello World # ভুল! কোডেশন শেষ করা হয়নি (SyntaxError).
```

অধ্যায় ২: ভ্যারিয়েবল ও ডেটা টাইপ (Variables & Data Types)

১. সহজ বাংলা ব্যাখ্যা

- **ভ্যারিয়েবল (Variable):** মনে করুন ভ্যারিয়েবল হলো রান্নাঘরের একটি প্লাস্টিকের কৌটা বা পানির বোতলের মতো। আপনি কৌটার গায়ে একটি লেবেল লাগিয়ে দিলেন "চিনি"। এখন এই কৌটার ভেতরে আপনি চিনি রাখতে পারেন, আবার চাইলে চিনি ফেলে দিয়ে লবণও রাখতে পারেন। পাইথনে ভ্যারিয়েবল হলো মেমোরির একটি নির্দিষ্ট জায়গার নাম, যেখানে আমরা ডেটা বা তথ্য জমা রাখি।
- **মেমোরি অ্যাসাইনমেন্ট ও রিঅ্যাসাইনমেন্ট:** `=` (ইকুয়াল) চিহ্নের মাধ্যমে ভ্যারিয়েবলে ডেটা রাখা হয়। একে বলে অ্যাসাইনমেন্ট। আপনি যদি একই ভ্যারিয়েবলে পরে নতুন কোনো ডেটা রাখেন, তবে আগের ডেটাটি মুছে গিয়ে নতুন ডেটা বসে যায়। একে বলে রিঅ্যাসাইনমেন্ট।
- **নামকরণের কঠোর নিয়মাবলী:**
 - নামের শুরুতে সংখ্যা (0-9) দেওয়া যাবে না।
 - নামের মধ্যে কোনো স্পেস বা ফাঁকা জায়গা থাকা যাবে না (স্পেসের বদলে আন্ডারস্কোর `_` ব্যবহার করতে হবে)।
 - পাইথনের নিজস্ব কিছু সংরক্ষিত শব্দ বা কিওয়ার্ড (যেমন: `if`, `for`, `print`) ভ্যারিয়েবলের নাম হিসেবে ব্যবহার করা যাবে না।
- **প্রধান ৪টি ডেটা টাইপ:**
 1. **String (str):** যেকোনো টেক্সট বা লেখা, যা কোটেশনের ভেতরে থাকে। (যেমন: `"Dhaka"`)
 2. **Integer (int):** যেকোনো পূর্ণসংখ্যা, পজিটিভ বা নেগেটিভ। (যেমন: `25`, `-5`)
 3. **Float (float):** যেকোনো ভগ্নাংশ বা দশমিক সংখ্যা। (যেমন: `3.1416`, `10.5`)
 4. **Boolean (bool):** শুধুমাত্র দুটি মান হতে পারে— `True` (সত্য) অথবা `False` (মিথ্যা)। (মনে রাখবেন, T এবং F বড় হাতের হবে)।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
# ভ্যারিয়েবল ডিক্লেয়ারেশন এবং অ্যাসাইনমেন্ট
user_age = 25          # Integer
product_price = 99.99  # Float
user_name = "Anik"     # String
is_logged_in = True    # Boolean

# রিঅ্যাসাইনমেন্ট মেকানিক্স
score = 10
print(score) # আউটপুট আসবে 10
```

```
score = 20    # মান পরিবর্তন করা হলো
print(score) # আউটপুট আসবে 20
```

৩. সঠিক ও ভুল কোডের উদাহরণ

● CORRECT DESIGN PATTERN

```
my_age = 22
total_score_1 = 550
is_python_easy = True
```

● INCORRECT DESIGN PATTERN

```
1st_name = "Rahat"    # ভুল! শুরুতেই সংখ্যা ব্যবহার করা হয়েছে (SyntaxError).
user name = "Sumon"   # ভুল! মাঝখানে স্পেস ব্যবহার করা হয়েছে।
if = 10               # ভুল! পাইথনের কিওয়ার্ড 'if' নাম হিসেবে ব্যবহার করা হয়েছে।
is_active = true      # ভুল! True-এর 't' ছোট হাতের দেওয়া হয়েছে (NameError).
```

অধ্যায় ৩: স্ট্রিং ও নাম্বার মেকানিক্স (String & Number Mechanics)

১. সহজ বাংলা ব্যাখ্যা

- **স্ট্রিং মেথডস:** পাইথনে স্ট্রিং বা টেক্সট নিয়ে খেলার জন্য কিছু চমৎকার বিল্ট-ইন `ツール` বা মেথড আছে।
 - `upper()` : পুরো লেখাকে বড় হাতের অক্ষরে রূপান্তর করে।
 - `lower()` : পুরো লেখাকে ছোট হাতের অক্ষরে রূপান্তর করে।
 - `title()` : প্রতিটি শব্দের প্রথম অক্ষর বড় হাতের করে দেয়।
- **হোয়াইটস্পেস ও এক্সেপ ক্যারেক্টার:** কোডের ভেতর নতুন লাইন বা ট্যাব তৈরি করতে বিশেষ চিহ্ন ব্যবহার করা হয়। `\n` দিলে পাইথন লেখাটিকে ভেঙে নতুন লাইনে নিয়ে যায় এবং `\t` দিলে ৪টি স্পেস সমপরিমাণ জায়গা (ট্যাব) তৈরি করে।
- **গাণিতিক অপারেটর ও সূচক:** যোগ (`+`), বিয়োগ (`-`), গুণ (`*`), ভাগ (`/`) ছাড়াও পাইথনে পাওয়ার বা স্কয়ার করার জন্য ডাবল স্টার (`**`) ব্যবহার করা হয় (যেমন: `2^3` লেখার জন্য `2 ** 3`)।
- **টাইপ কনভার্সন (Type Conversion):** পাইথনে আপনি সরাসরি একটি স্ট্রিং-এর সাথে একটি নাম্বার যোগ করতে পারবেন না। যেমন: `"বইয়ের দাম " + 150` লিখলে পাইথন রেগে যাবে। তাই নাম্বারটিকে স্ট্রিং বানিয়ে নিতে হয় `str(150)` লিখে। একেই বলে টাইপ কনভার্সন।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
# স্ট্রিং মেথডস
text = "hello bangladesh"
print(text.title()) # আউটপুট: Hello Bangladesh

# এক্সেপ ক্যারেক্টার
print("লাইন ১
লাইন ২")

# গাণিতিক অপারেটর ও টাইপ কনভার্সন
price = 250
message = "টোটাল বিল হলো: " + str(price) + " টাকা"
print(message)
```

৩. সঠিক ও ভুল কোডের উদাহরণ

● CORRECT DESIGN PATTERN

```
calculation = 5 ** 2 # ২৫  
greeting = "Hello " + "World" # দুটি স্ট্রিং যোগ (Concatenation)  
print(greeting)
```

● INCORRECT DESIGN PATTERN

```
age = 20  
print("আমার বয়স " + age) # ভুল! String এবং Integer সরাসরি যোগ হয় না (TypeError).
```

অধ্যায় ৪: পাইথনের পেশী (Lists & Loops)

১. সহজ বাংলা ব্যাখ্যা

- **লিস্ট (List):** একটি ভ্যারিয়েবলে আমরা কেবল একটি ডেটা রাখতাম। কিন্তু একই ব্র্যাকেটের ভেতর কমা দিয়ে আলাদা করে যখন অনেকগুলো ডেটা একসাথে রাখা হয়, তাকে লিস্ট বলে। একে পাইথনের থার্ড ব্র্যাকেট `[]` দিয়ে প্রকাশ করা হয়।
- **ইনডেক্সিং ও নেগেটিভ ইনডেক্সিং:** লিস্টের প্রতিটা উপাদানের একটা পজিশন বা রোল নাম্বার থাকে, যাকে ইনডেক্স বলে। পাইথনে গণনা শুরু হয় `0` থেকে। অর্থাৎ, ১ম উপাদানের ইনডেক্স `0`, ২য় উপাদানের ইনডেক্স `1` ইত্যাদি। আবার উল্টো দিক (ডান দিক) থেকে গণনা করতে চাইলে নেগেটিভ ইনডেক্স ব্যবহার করা হয়। একদম শেষের উপাদানের ইনডেক্স হলো `-1`, তার আগেরটি `-2` ইত্যাদি।
- **মিউটেবিলিটি (Mutability):** লিস্টের ভেতরের যেকোনো উপাদানকে যেকোনো সময় বদলে ফেলা যায়। এই পরিবর্তন করার ক্ষমতাকে 'মিউটেবিলিটি' বলে।
- **আইটেম হ্যান্ডলিং:**
 - `append()` : লিস্টের একদম শেষে নতুন উপাদান যোগ করে।
 - `insert()` : লিস্টের নির্দিষ্ট যেকোনো ইনডেক্সে নতুন উপাদান ঢুকিয়ে দেয়।
 - `del` : ইনডেক্স ধরে চিরতরে উপাদান মুছে দেয়।
 - `pop()` : লিস্টের শেষ উপাদানটি কেটে বের করে আনে (যা পরে অন্য কোথাও ব্যবহার করা যায়)।
 - `remove()` : ইনডেক্স না জানলে সরাসরি উপাদানের নাম ধরে মুছে দেয়।
- **স্বায়ী ও সাময়িক সর্টিং:** `sort()` মেথড ব্যবহার করলে লিস্টের উপাদানগুলো চিরতরে এ-জেড (A-Z) বা ক্রমানুসারে সাজানো হয়ে যায়। আর `sorted()` ফাংশন ব্যবহার করলে মূল লিস্ট ঠিক রেখে সাময়িকভাবে সাজানো আউটপুট দেখায়।
- **for লুপ ও ইন্ডেন্টেশন:** লিস্টের ১টি ১টি করে উপাদান ধরে আলাদাভাবে কাজ করার জন্য `for` লুপ ব্যবহার করা হয়। লুপের ভেতরের কোডগুলোকে ৪টি স্পেস ডানে সরিয়ে লিখতে হয়, একে বলে **ইন্ডেন্টেশন (Indentation)**। পাইথনে ব্র্যাকেটের বালাই নেই, এই ইন্ডেন্টেশন দিয়েই কোডের ব্লক চেনা যায়।
- **range() সিকোয়েন্স:** ধারা বা স্বয়ংক্রিয়ভাবে নাম্বারের সিকোয়েন্স তৈরি করতে `range()` ব্যবহার করা হয়।
- **ট্যাপলস (Tuples):** এটি দেখতে লিস্টের মতোই, কিন্তু একে ফার্স্ট ব্র্যাকেট `()` দিয়ে লিখতে হয়। ট্যাপলস হলো **ইমিউটেবল (Immutable)**, অর্থাৎ একবার তৈরি করলে এর ভেতরের উপাদান আর কখনো পরিবর্তন, যোগ বা বিয়োগ করা যায় না।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
# লিস্ট তৈরি এবং ইনডেক্সিং
fruits = ["Mango", "Banana", "Apple"]
print(fruits[0]) # আউটপুট: Mango
print(fruits[-1]) # আউটপুট: Apple (সর্বশেষ উপাদান)

# আইটেম হ্যান্ডলিং
fruits.append("Orange") # শেষে যোগ হবে
fruits.insert(1, "Grape") # ১ নম্বর ইনডেক্সে (দ্বিতীয় স্থানে) Grape বসবে
popped_fruit = fruits.pop() # Orange কেটে বের করে আনবে

# for লুপ এবং range()
for number in range(1, 4): # ১ থেকে শুরু হয়ে ৪ এর ঠিক আগে (৩ পর্যন্ত) চলবে
    print(number) # ইন্ডেন্টেশন (৪টি স্পেস ডানে)
```

৩. সঠিক ও ভুল কোডের উদাহরণ

CORRECT DESIGN PATTERN

```
dimensions = (200, 50) # এটি একটি টাপল
print(dimensions[0]) # আউটপুট: 200

colors = ["red", "green"]
for color in colors:
    print(color) # সঠিক ইন্ডেন্টেশন
```

INCORRECT DESIGN PATTERN

```
# ভুল ১: টাপলের মান পরিবর্তনের চেষ্টা
dimensions = (200, 50)
dimensions[0] = 250 # ভুল! (TypeError: 'tuple' object does not support item assignment)

# ভুল ২: ইন্ডেন্টেশন এরর
for x in range(3):
    print(x) # ভুল! লুপের ভেতরের কোড ডানে সরানো হয়নি (IndentationError).
```

অধ্যায় ৫: সিদ্ধান্ত গ্রহণ ও ডিকশনারি (Decisions & Dictionaries)

১. সহজ বাংলা ব্যাখ্যা

- **তুলনামূলক অপারেটর (= বনাম ==):** মনে রাখুন, একক ইকুয়াল = হলো কোনো ভ্যারিয়েবলে মান রাখার (Assign করার) জন্য। আর ডাবল ইকুয়াল == হলো দুই পাশের মান সমান কিনা তা পরীক্ষা বা তুলনা করার জন্য। এছাড়া বড় (>), ছোট (<), অসমান (!=) অপারেটরও ব্যবহার করা হয়।
- **if-elif-else গেট মেকানিজম:** এটি হলো কোডের ট্রাফিক সিগন্যাল। if (যদি) এর কন্ডিশন সত্য হলে কোড সেই গেট দিয়ে চুকে যায়। যদি তা মিথ্যা হয়, তবে সে elif (নাহলে যদি) গেট চেক করে। সব গেট মিথ্যা হলে সবশেষে else (নাহলে) ব্লকের কাজ সম্পন্ন হয়।
- **লজিক্যাল অপারেটর (and, or, not):**
 - and : যখন দুটি শর্তই সত্য হতে হবে।
 - or : যেকোনো একটি শর্ত সত্য হলেই চলবে।
 - not : সত্যকে মিথ্যা আর মিথ্যাকে সত্যে উল্টে দেওয়ার জন্য।
- **মেম্বারশিপ টেস্টিং (in, not in):** কোনো উপাদান একটি লিস্ট বা কালেকশনের ভেতরে আছে কিনা তা সরাসরি চেক করার জন্য in বা not in ব্যবহার করা হয়।
- **ডিকশনারি (Dictionary):** বাস্তব জীবনের অভিধানের মতো, যেখানে একটি শব্দের বিপরীতে তার অর্থ থাকে। পাইথনে একে কৌকড়ানো ব্র্যাকেট {} দিয়ে লেখা হয় এবং এখানে প্রতিটা ডাটা **Key-Value** জোড়া হিসেবে থাকে। যেমন: "name": "Anik"। এখানে "name" হলো Key আর "Anik" হলো Value।
- **ডাটা লুপ করা:** ডিকশনারি থেকে শুধু Key পেতে .keys() , শুধু Value পেতে .values() , এবং Key ও Value একসাথে পেতে .items() মেথড দিয়ে লুপ চালাতে হয়।
- **নিরাপদ get() মেথড:** ডিকশনারিতে কোনো Key যদি না থাকে আর আপনি যদি তা সরাসরি খোঁজেন (যেমন: user['age']), তবে কোড ক্র্যাশ করবে। এই ক্র্যাশ এড়াতে .get() মেথড ব্যবহার করা হয়; Key না থাকলে এটি ক্র্যাশ না করে শান্তভাবে None বা আপনার দেওয়া একটি ডিফল্ট মেসেজ রিটার্ন করে।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
# if-elif-else মেকানিক্স
score = 85
if score >= 80 and score <= 100:
    print("A+")
elif score >= 40:
```

```

    print("Pass")
else:
    print("Fail")

# ডিকশনারি মেকানিজম
student = {"name": "Tamim", "score": 95}
print(student["name"]) # আউটপুট: Tamim

# নিরাপদ get() ব্যবহার
print(student.get("age", "এই তথ্যটি নেই")) # ক্র্যাশ করবে না, ডিফল্ট টেক্সট দেখাবে।

```

৩. সঠিক ও ভুল কোডের উদাহরণ

● CORRECT DESIGN PATTERN

```

alien = {"color": "green"}
if alien["color"] == "green": # তুলনা করার জন্য == ব্যবহার সঠিক
    print("You earned 5 points.")

```

● INCORRECT DESIGN PATTERN

```

# ভুল ১: শর্তে অ্যাসাইনমেন্ট অপারেটর ব্যবহার
age = 18
if age = 18: # ভুল! শর্তের ভেতর '=' হবে না, '==' হবে (SyntaxError).
    print("Adult")

# ভুল ২: ডিকশনারিতে সরাসরি অনুপস্থিত Key খোঁজা
user = {"name": "Asif"}
print(user["email"]) # ভুল! 'email' নামের কোনো Key নেই (KeyError).

```

অধ্যায় ৬: ইন্টারঅ্যাক্টিভিটি ও পুনঃব্যবহারযোগ্যতা (Interactivity & Reusability)

১. সহজ বাংলা ব্যাখ্যা

- **ইউজার ইনপুট ও টাইপ কনভার্সন:** ইউজারের কাছ থেকে সরাসরি কোনো তথ্য নেওয়ার জন্য `input()` ফাংশন ব্যবহার করা হয়। মনে রাখবেন, `input()` দিয়ে পাইথন যা-ই গ্রহণ করুক না কেন, সেটিকে সে সর্বদাই **String (লেখা)** হিসেবে ধরে নেয়। তাই ইউজারের কাছ থেকে যদি বয়স বা কোনো সংখ্যা নিতে চান, তবে তাকে `int()` বা `float()` দিয়ে জোরপূর্বক নাম্বারে রূপান্তর বা কনভার্ট করে নিতে হয়।
- **while লুপ ও ইনফিনিট লুপ:** যতক্ষণ পর্যন্ত একটি শর্ত সত্য থাকবে, ততক্ষণ `while` লুপ চলতেই থাকবে। যদি শর্তটি কোনোদিন মিথ্যা না হয়, তবে লুপটি কেয়ামত পর্যন্ত চলতেই থাকবে, যাকে **ইনফিনিট লুপ (Infinite Loop)** বলে। এটি কম্পিউটারকে হ্যাং বা ক্র্যাশ করিয়ে দিতে পারে।
- **লুপ কন্ট্রোল (break, continue):**
 - `break` : লুপের মেয়াদ শেষ হওয়ার আগেই লুপকে লাথি মেরে তাৎক্ষণিকভাবে বন্ধ করে দেওয়ার জন্য।
 - `continue` : লুপের বর্তমান চক্রটি মাঝপথে বাদ দিয়ে সরাসরি পরের চক্রে চলে যাওয়ার জন্য।
- **বুলিয়ান ফ্ল্যাগস (Boolean Flags):** একটি ভ্যারিয়েবলে `True` বা `False` রেখে সেটিকে পুরো প্রোগ্রামের সিগন্যাল বা সুইচ হিসেবে ব্যবহার করা।
- **মডুলো (%) অপারেটর:** এটি কোনো সংখ্যাকে ভাগ করার পর তার **ভাগশেষ** কত থাকে তা বের করে দেয় (যেমন: `7 % 2` এর ফলাফল হবে `1`)।
- **ফাংশন ডেফিনিশন ও কল:** একই কোড বারবার না লিখে একটি বাক্সে বন্দি করে রাখার নাম ফাংশন। `def` কিওয়ার্ড দিয়ে ফাংশন তৈরি (Define) করতে হয় এবং পরে নাম ধরে ডাকলে (Call) সেটি রান হয়।
- **প্যারামিটার বনাম আর্গুমেন্ট:** ফাংশন তৈরি করার সময় ব্রাকেটের ভেতরের ভ্যারিয়েবলগুলোকে বলে **প্যারামিটার** (যেমন: নীল নকশা বা ব্লুপ্রিন্ট)। আর ফাংশনটি রান করার সময় আসল যে ডেটা পাঠানো হয়, তাকে বলে **আর্গুমেন্ট**।
- **print() বনাম return এর গভীর পার্থক্য:** এটি নতুনদের সবচেয়ে বেশি ট্রিপ বা কনফিউজড করে। `print()` শুধু স্ক্রিনে একটা লেখা দেখায়, কিন্তু সেই মানটি অন্য কোনো ভ্যারিয়েবলে জমা রাখা বা পরবর্তীতে ব্যবহার করা যায় না। আর `return` হলো ফাংশনের কাজ শেষে একটি রেজাল্ট বা ডাটা হাতেনাতে ফেরত দেওয়া, যা দিয়ে পরবর্তীতে অন্য কোনো গাণিতিক বা লজিক্যাল কাজ করা যায়।
- **ভ্যারিয়েবল স্কোপ (Local vs Global):** ফাংশনের ভেতরে তৈরি করা ভ্যারিয়েবল হলো **Local**, এটি ফাংশনের বাইরে থেকে চেনা যায় না। আর ফাংশনের বাইরে একদম শুরুতে তৈরি করা ভ্যারিয়েবল হলো **Global**, এটি যেকোনো জায়গা থেকে ব্যবহার করা যায়।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
# ইনপুট এবং টাইপ কনভার্সন
age_input = input("আপনার বয়স দিন: ")
age = int(age_input) # নাম্বারে কনভার্ট করা হলো

# while লুপ এবং break
count = 1
while count <= 5:
    if count == 3:
        break # ৩ এ পৌঁছালে লুপ এখানেই শেষ
    print(count)
    count += 1

# ফাংশন এবং return
def add_numbers(num1, num2): # num1, num2 হলো প্যারামিটার
    return num1 + num2      # মান ফেরত পাঠানো হলো

result = add_numbers(10, 20) # ১০, ২০ হলো আর্গুমেন্ট
print(result)                # আউটপুট: 30
```

৩. সঠিক ও ভুল কোডের উদাহরণ

CORRECT DESIGN PATTERN

```
def calculate_square(number):
    return number * number

output = calculate_square(5) # output এর ভেতর ২৫ জমা হলো
print(output)
```

INCORRECT DESIGN PATTERN

```
# ভুল ১: input() কে সরাসরি নাম্বারের সাথে তুলনা করা
user_age = input("Enter age: ") # ইউজার দিলো "18" (যা একটি স্ট্রিং)
if user_age >= 18:                # ভুল! String এবং Integer তুলনা হয় না (TypeError).
    print("You can vote.")

# ভুল ২: print-কে return মনে করা
def power_two(x):
```

```
print(x * x)
```

```
final_res = power_two(4) + 10    # ভুল! power_two কিছু return করেনি, তাই এটি None! (TypeError)
```



পার্ট ২: প্রফেশনাল ও স্ট্রাকচার্ড পাইথন (OOP & Operations)

অধ্যায় ১: অবজেক্ট-ওরিয়েন্টেড প্রোগ্রামিং (OOP)

১. সহজ বাংলা ব্যাখ্যা

- **বাস্তব জগতকে কোডে রূপ দেওয়া:** আমরা এতক্ষণ ছোট ছোট লজিক ও ভ্যারিয়েবল নিয়ে কাজ করেছি। কিন্তু বাস্তব জগতের বড় বড় প্রজেক্টের জিনিসগুলোকে (যেমন: একটি গাড়ি, একজন ব্যাংক কাস্টমার, বা গেমের একটা ক্যারেক্টার) কোডে সহজে প্রকাশ করার পদ্ধতির নামই হলো অবজেক্ট-ওরিয়েন্টেড প্রোগ্রামিং বা OOP।
- **ক্লাস (Class) বনাম অবজেক্ট (Object/Instance):**
 - **ক্লাস** হলো একটি নকশা বা ব্লুপ্রিন্ট (Blueprint)। যেমন: একটি বাড়ি তৈরির ইঞ্জিনিয়ারিং নকশা। এই নকশাটি নিজে কিন্তু কোনো বাড়ি নয়, জাস্ট একটা প্ল্যান।
 - **অবজেক্ট** হলো সেই নকশা দেখে তৈরি করা আসল বাড়িটি। পাইথনে ক্লাস থেকে যখন বাস্তব কিছু তৈরি করা হয়, তাকে অবজেক্ট বা ইনস্ট্যান্স (Instance) বলে। একটি ক্লাস বা নকশা দিয়ে হাজার হাজার অবজেক্ট তৈরি করা সম্ভব।
- **self নামক ম্যাজিক:** যখন আপনি ক্লাস থেকে একাধিক অবজেক্ট তৈরি করবেন, তখন পাইথন কীভাবে বুঝবে আপনি কোন অবজেক্টের ডাটা নিয়ে কাজ করছেন? এই কনফিউশন দূর করার জন্য পাইথনে `self` ব্যবহার করা হয়। `self` মানে হলো "এই নির্দিষ্ট অবজেক্টটি"। এটি ক্লাসের ভেতরের ফাংশন বা মেথডগুলোর ১ম প্যারামিটার হিসেবে বাধ্যতামূলকভাবে বসে।
- **কনস্ট্রাক্টর (__init__):** এটি ক্লাসের ভেতরের একটি স্পেশাল ম্যাজিক মেথড। ক্লাস থেকে যখনই নতুন কোনো অবজেক্ট তৈরি করা হয়, এই `__init__` মেথডটি ব্যাকগ্রাউন্ডে স্বয়ংক্রিয়ভাবে বা নিজে নিজেই রান হয়ে যায়। এর প্রধান কাজ হলো অবজেক্ট তৈরি হওয়ার সাথে সাথে তার প্রাথমিক ডাটা বা বৈশিষ্ট্যগুলো সেট করে দেওয়া।
- **অ্যাট্রিবিউট (Attribute) বনাম মেথড (Method):**
 - **অ্যাট্রিবিউট:** অবজেক্টের বৈশিষ্ট্য বা ডাটা (যেমন: গাড়ির কালার, মডেল, স্পিড)। এগুলো আসলে ক্লাসের ভেতরের ভ্যারিয়েবল।
 - **মেথড:** অবজেক্টের আচরণ বা কাজ (যেমন: গাড়ি স্টার্ট হওয়া, ব্রেক করা)। এগুলো আসলে ক্লাসের ভেতরের ফাংশন।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
# ক্লাস তৈরি (কনভেনশন অনুযায়ী ক্লাসের নামের প্রথম অক্ষর বড় হাতের হয়)
class Car:
    # কনস্ট্রাক্টর মেথড - প্রাথমিক ডাটা সেটের জন্য
    def __init__(self, brand, model):
        self.brand = brand      # এটি একটি অ্যাট্রিবিউট
        self.model = model      # এটিও একটি অ্যাট্রিবিউট
        self.speed = 0          # ডিফল্ট অ্যাট্রিবিউট

    # এটি একটি মেথড (আচরণ)
    def accelerate(self):
        self.speed += 10
        print(f"{self.brand} গাড়িটির বর্তমান গতি {self.speed} কিমি/ঘণ্টা।")

# ক্লাস থেকে অবজেক্ট বা ইনস্ট্যান্স তৈরি
my_car = Car("Toyota", "Corolla") # এখানে self ব্যাকগ্রাউন্ডে নিজে কাজ করে
another_car = Car("Tesla", "Model 3")

# মেথড কল করা
my_car.accelerate() # আউটপুট: Toyota গাড়িটির বর্তমান গতি 10 কিমি/ঘণ্টা।
```

৩. সঠিক ও ভুল কোডের উদাহরণ

CORRECT DESIGN PATTERN

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        print(f"{self.name} ঘেউ ঘেউ করছে!")

my_dog = Dog("Tommy")
my_dog.bark()
```

INCORRECT DESIGN PATTERN

```
class Dog:
    def __init__(name): # ভুল! প্রথম প্যারামিটার হিসেবে 'self' দেওয়া হয়নি।
        self.name = name
```

```
def bark(self):  
    print(f"{self.name} is barking") # (NameError বা TypeError আসবে)
```

অধ্যায় ২: অ্যাডভান্সড OOP কনসেপ্ট (Advanced OOP)

১. সহজ বাংলা ব্যাখ্যা

- **ইনহেরিটেন্স বা উত্তরাধিকার (Inheritance):** মনে করুন আপনার বাবা একটি চমৎকার বাড়ি বানিয়েছেন। সন্তান হিসেবে আপনি সেই বাড়ির মালিকানা স্বয়ংক্রিয়ভাবেই পেয়ে যাচ্ছেন, প্লাস আপনি চাইলে সেই বাড়িতে নতুন কোনো রুমও যোগ করতে পারেন। প্রোগ্রামিংয়েও ঠিক তাই! একটি পুরনো ক্লাসের (Parent Class) সমস্ত বৈশিষ্ট্য ও মেথড নতুন একটি ক্লাসে (Child Class) কপি-পেস্ট না করে সরাসরি ধার বা ব্যবহার করার নামই হলো ইনহেরিটেন্স।
- **মেথড ওভাররাইডিং (Method Overriding):** প্যারেন্ট ক্লাসের কোনো মেথড বা আচরণ যদি চাইল্ড ক্লাসের পছন্দ না হয়, তবে চাইল্ড ক্লাস চাইলে নিজের ভেতরে হুবহু একই নামের মেথড তৈরি করে তার ভেতরের লজিক সম্পূর্ণ বদলে দিতে পারে। একে বলে মেথড ওভাররাইডিং।
- **কম্পোজিশন (Composition):** ইনহেরিটেন্সে সম্পর্কটা হয় "Is-A" (যেমন: Car is a Vehicle)। আর কম্পোজিশনে সম্পর্কটা হয় "Has-A" (যেমন: Car has an Engine)। অর্থাৎ, একটি বড় ক্লাসের ভেতরে সরাসরি অন্য একটি ক্লাসের অবজেক্টকে একটি অ্যাট্রিবিউট বা উপাদান হিসেবে রেখে দেওয়ার নামই হলো কম্পোজিশন। এটি কোডকে অনেক বেশি মডিউলার বা পরিচ্ছন্ন করে।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
# প্যারেন্ট ক্লাস (Parent Class)
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

    def start_engine(self):
        print("ইঞ্জিন চালু হলো!")

# চাইল্ড ক্লাস (Child Class) যা Vehicle-কে ইনহেরিট করছে
class ElectricCar(Vehicle):
    def __init__(self, brand, battery_size):
        super().__init__(brand) # super() দিয়ে প্যারেন্ট ক্লাসের __init__ কে কল করা হয়
        self.battery_size = battery_size

    # মেথড ওভাররাইডিং (প্যারেন্টের মেথড বদলে দেওয়া)
    def start_engine(self):
        print("কোনো শব্দ ছাড়াই ইলেকট্রিক ইঞ্জিন নিঃশব্দে চালু হলো!")

# কম্পোজিশন (Composition) এর উদাহরণ
class Engine:
    def __init__(self, horsepower):
```

```

        self.horsepower = horsepower

class SportsCar:
    def __init__(self, brand, hp):
        self.brand = brand
        self.engine = Engine(hp) # SportsCar ক্লাসের ভেতর Engine ক্লাসের অবজেক্ট!

tesla = ElectricCar("Tesla", "100 kWh")
tesla.start_engine() # আউটপুট চাইল্ড ক্লাস থেকে আসবে (ওভাররাইড মেকানিজম)

```

৩. সঠিক ও ভুল কোডের উদাহরণ

● CORRECT DESIGN PATTERN

```

class Animal:
    def speak(self):
        return "Some sound"

class Cat(Animal):
    def speak(self): # সঠিকভাবে ওভাররাইড করা হয়েছে
        return "Meow"

```

● INCORRECT DESIGN PATTERN

```

class Parent:
    def __init__(self, name):
        self.name = name

class Child(Parent):
    def __init__(self, name, age):
        # ভুল! প্যারেন্ট ক্লাসের __init__ কে super() দিয়ে ইনিশিয়ালাইজ করা হয়নি।
        self.age = age

ch = Child("Anik", 12)
print(ch.name) # ভুল! (AttributeError: 'Child' object has no attribute 'name')

```

অধ্যায় ৩: পারসিস্টেন্ট স্টোরেজ ও এরর ডিফেন্স (Files & Exceptions)

১. সহজ বাংলা ব্যাখ্যা

- **ফাইল অপারেশন ও with ব্লক:** কোড রান করা অবস্থায় ভ্যারিয়েবলে ডাটা রাখলে প্রোগ্রাম বন্ধ করার সাথে সাথে ডাটা ভ্যানিশ হয়ে যায়। ডা খানকে কম্পিউটারে স্থায়ীভাবে বা পারসিস্টেন্টলি সেভ করার জন্য আমরা টেক্সট বা JSON ফাইল ব্যবহার করি। পাইথনে ফাইল ওপেন করার জন্য `with open(...)` ব্লক ব্যবহার করা সবচেয়ে নিরাপদ। কারণ কাজ শেষ হওয়া মাত্র পাইথন স্বয়ংক্রিয়ভাবে ফাইলটি ক্লোজ করে দেয়, মেমোরি লিক হওয়ার ভয় থাকে না।
- **রিড, রাইট ও অ্যাপেন্ড মোড:**
 - `'r'` (Read): ফাইল থেকে ডাটা শুধু পড়ার জন্য। ফাইল না থাকলে এরর আসবে।
 - `'w'` (Write): ফাইলে নতুন ডাটা লেখার জন্য। মনে রাখবেন, এই মোড ফাইল আগের সব লেখা মুছে একদম নতুন করে লেখে!
 - `'a'` (Append): ফাইলের আগের লেখা ঠিক রেখে তার একদম নিচে নতুন লেখা বা ডাটা জোড়া দেওয়ার জন্য।
- **এক্সেপশন হ্যান্ডলিং (এরর ডিফেন্স):** কোডে যেকোনো সময় ভুল ইনপুট বা সার্ভার ডাউন হওয়ার কারণে এরর আসতে পারে (যেমন: ০ দিয়ে কোনো কিছুকে ভাগ করা বা কম্পিউটারে ফাইল না থাকা)। এই এররগুলোর কারণে পুরো সফটওয়্যার যেন ক্র্যাশ না করে, সেজন্য আমরা `try-except` ডিফেন্স ওয়াল ব্যবহার করি।
- **try / except / else / finally এর লাইফসাইকেল:**
 - `try` : যে কোডে এরর আসার সম্ভাবনা আছে, তা এখানে রাখা হয়।
 - `except` : ট্রাই ব্লকে এরর আসলে এই ব্লকটি সচল হয় এবং ক্র্যাশ থেকে বাঁচায়।
 - `else` : ট্রাই ব্লকে যদি কোনো এরর **না আসে**, তবেই কেবল এটি রান হয়।
 - `finally` : এরর আসুক আর না আসুক, এই ব্লকের কোড **অবশ্যই রান হবে** (যেমন: ডাটাবেজ কানেকশন ক্লোজ করা)।
- **JSON রিড-রাইট:** পাইথনের ডিকশনারি বা লিস্টকে সরাসরি ফাইলে টেক্সট হিসেবে রাখলে পরবর্তীতে তা ব্যবহারে অসুবিধা হয়। তাই স্ট্রাকচার্ড ডাটা সেভ করার আন্তর্জাতিক মাধ্যম হলো JSON। পাইথনের ডাটা ফাইলে সেভ করতে `json.dump()` এবং ফাইল থেকে পাইথনে ব্যাক করতে `json.load()` ব্যবহার করা হয়।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
import json
```

```
# ফাইল রাইট এবং এক্সেপশন হ্যান্ডলিং এর কন্ট্রোল মেকানিজম
data_to_save = {"name": "Sajid", "role": "Developer"}

try:
    with open("user_data.json", "w") as file:
        json.dump(data_to_save, file) # JSON ফাইলে রাইট করা হলো
    print("ফাইল সফলভাবে সেভ হয়েছে।")
except IOError:
    print("ফাইল রাইট করার সময় কোনো একটি সমস্যা হয়েছে!")
finally:
    print("ফাইল অপারেশনের কাজ শেষ। ")
```

৩. সঠিক ও ভুল কোডের উদাহরণ

CORRECT DESIGN PATTERN

```
try:
    num = int(input("একটি সংখ্যা দিন: "))
    result = 10 / num
except ZeroDivisionError:
    print("০ দিয়ে ভাগ করা সম্ভব না!")
except ValueError:
    print("দয়া করে শুধু পূর্ণসংখ্যা দিন।")
```

INCORRECT DESIGN PATTERN

```
# ভুল ১: open করলে close না করা (with ছাড়া)
file = open("demo.txt", "w")
file.write("Hello")
# file.close() করতে ভুলে গেলে ফাইলটি মেমোরিতে লক হয়ে থাকতে পারে।

# ভুল ২: নগ্ন বা জেনেরিক except ব্যবহার করা
try:
    res = 5 / 0
except: # ভুল! কোন এরর তা স্পষ্ট না করলে আসল সমস্যা ধরা কঠিন হয়ে যায়।
    print("Something went wrong")
```

অধ্যায় ৪: ইন্ডাস্ট্রি স্ট্যান্ডার্ড কোড টেস্টিং (Testing Your Code)

১. সহজ বাংলা ব্যাখ্যা

- **অনুমানের বদলে প্রমাণ করার মানসিকতা:** আমরা কোড লেখার পর সাধারণত নিজে নিজে দুই-একটি ইনপুট দিয়ে টেস্ট করি যে "হ্যাঁ, কোড তো কাজ করছে!"। কিন্তু ইন্ডাস্ট্রি লেভেলে বড় সফটওয়্যারে হাজার হাজার ফাংশন থাকে। একটি ফাংশন পরিবর্তন করলে অন্যটি ভেঙে গেল কিনা তা হাত দিয়ে চেক করা অসম্ভব। তাই কোড ঠিক আছে কিনা তা প্রমাণ করার জন্য আমরা আলাদা **অটোমেটেড টেস্টিং কোড** লিখি।
- **ইউনিট টেস্ট ও unittest মডিউল:** পাইথনে কোডের প্রতিটি ছোট অংশ বা একক ফাংশন আলাদাভাবে পরীক্ষা করার জন্য বিল্ট-ইন `unittest` মডিউল ব্যবহার করা হয়।
- **অ্যাসারশনস (Assertions):** এটি হলো টেস্ট কোডের আসল হাতিয়ার। এর মাধ্যমে আমরা পাইথনকে বলি — "আমি আশা করছি এই ফাংশনের আউটপুট হবে ২৫, তুমি চেক করে দেখ আসল আউটপুট ২৫ এসেছে কিনা।"
 - `assertEqual(a, b)` : চেক করে a এবং b সমান কিনা।
 - `assertTrue(x)` : চেক করে শর্তটি `True` কিনা।
 - `assertRaises(Error)` : চেক করে নির্দিষ্ট কোনো ভুলের কারণে কোডে এরর আসছে কিনা।
- **test_ নামকরণের কঠোর নিয়ম:** পাইথনের টেস্ট রানার স্বয়ংক্রিয়ভাবে টেস্ট ফাইল ও ফাংশন খুঁজে বের করে। এর জন্য একটি কঠোর নিয়ম হলো—আপনার টেস্ট ফাইলের নাম এবং ফাইলের ভেতরের টেস্ট মেথড বা ফাংশনগুলোর নামের শুরুতে অবশ্যই `test_` কথাটি থাকতে হবে (যেমন: `test_addition()`)। অন্যথায় পাইথন সেটিকে টেস্ট হিসেবে গণ্যই করবে না।

২. কোডের সিনট্যাক্স ও মেকানিক্স

ধরে নিন আপনার একটি আসল কোডের ফাইল আছে যার নাম `math_utils.py` :

```
# math_utils.py (মূল কোড)
def get_cube(number):
    return number ** 3
```

এখন এই ফাংশনটি টেস্ট করার জন্য আমরা আরেকটি ফাইল তৈরি করব যার নাম `test_math_utils.py` :

```
# test_math_utils.py (টেস্ট কোড)
import unittest
from math_utils import get_cube # মূল ফাংশন ইমপোর্ট করা হলো

class TestMathFunctions(unittest.TestCase): # TestCase-কে ইনহেরিট করতে হবে
```

```
def test_cube_calculation(self): # নামের শুরুতে test_ বাধ্যতামূলক
    result = get_cube(3)
    self.assertEqual(result, 27) # আমরা জানি ৩ এর কিউব ২৭, তাই চেক করছি

if __name__ == "__main__":
    unittest.main()
```

৩. সঠিক ও ভুল কোডের উদাহরণ

CORRECT DESIGN PATTERN

```
# ফাংশন নামের শুরুতে test_ দেওয়া হয়েছে
def test_user_status(self):
    self.assertTrue(is_admin("rahim"))
```

INCORRECT DESIGN PATTERN

```
# ভুল! ফাংশনের শুরুতে test_ দেওয়া হয়নি, তাই এই টেস্টটি রানই হবে না!
class MyTest(unittest.TestCase):
    def check_my_function(self):
        self.assertEqual(add(1,2), 3)
```



পার্ট ৩: অ্যাডভান্সড পাইথন ও মেমোরি মেকানিক্স

অধ্যায় ১: মেমোরি ও মিউটেবিলিটি ডিপ ডাইভ (Memory & Mutability Deep Dive)

১. সহজ বাংলা ব্যাখ্যা

- **পাইথনে মেমোরি ম্যানেজমেন্ট:** পাইথনে সবকিছুই আসলে এক একটি **Object**। আমরা যখন লিখি `x = 5`, তখন পাইথন মেমোরিতে (RAM) `5` নামের একটি অবজেক্ট তৈরি করে এবং `x` ভ্যারিয়েবলটিকে সেই অবজেক্টের দিকে তাক করে বা রেফারেন্স (Reference) দিয়ে রাখে।
- **ID এবং is বনাম == এর আসল পার্থক্য:**
 - `id()` ফাংশন দিয়ে মেমোরিতে কোনো অবজেক্টের আসল ঠিকানা বা অ্যাড্রেস জানা যায়।
 - `==` অপারেটর কেবল দুই পাশের মান (Value) সমান কিনা তা পরীক্ষা করে।
 - `is` অপারেটর পরীক্ষা করে দুই পাশের ভ্যারিয়েবল মেমোরির একই ঠিকানায় (Same Identity) বসে আছে কিনা।
- **মেমোরির বিপজ্জনক বাগ ([[0]*3]*3):** নতুনরা ২D বা টু-ডাইমেনশনাল ম্যাট্রিক্স বানাতে গিয়ে এই ভুলটি প্রায়ই করেন। পাইথনে যখন আপনি `[[0]*3]*3` লিখবেন, পাইথন মেমোরিতে ১টি মাত্র ভেতরের লিস্ট `[0, 0, 0]` তৈরি করে এবং সেটির ৩টি হুবহু মেমোরি কপি বা রেফারেন্স মেইন লিস্টে বসিয়ে দেয়। ফলে আপনি যদি যেকোনো ১টি রো (Row)-এর মান পরিবর্তন করেন, দেখবেন জাদুর মতো বাকি ৩টি রো-এর মানও একসাথে বদলে গেছে!
- **Shallow Copy বনাম Deep Copy:**
 - **Shallow Copy:** এটি একটি নতুন কালেকশন তৈরি করে, কিন্তু ভেতরের নেস্টেড বা চাইল্ড অবজেক্টগুলোর আসল মেমোরি রেফারেন্সই রেখে দেয়।
 - **Deep Copy:** এটি মূল অবজেক্টের ভেতরের একদম শেষ প্রান্ত পর্যন্ত প্রতিটা ডাটার সম্পূর্ণ নতুন ও স্বাধীন মেমোরি কপি তৈরি করে।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
import copy

# is বনাম == এর মেকানিক্স
list1 = [1, 2, 3]
list2 = [1, 2, 3]

print(list1 == list2) # True (কারণ ভেতরের উপাদান বা মান সমান)
print(list1 is list2) # False (কারণ মেমোরিতে এরা দুটি আলাদা জায়গায় তৈরি হয়েছে)

# Shallow বনাম Deep Copy মেকানিক্স
original = [[1, 2], [3, 4]]
shallow_res = copy.copy(original)
deep_res = copy.deepcopy(original)

original[0][0] = 99
print(shallow_res[0][0]) # আউটপুট: 99 (মূল ডাটা বদলালে Shallow copy ও বদলে যায়!)
print(deep_res[0][0]) # আউটপুট: 1 (Deep copy সম্পূর্ণ সুরক্ষিত ও স্বাধীন!)
```

৩. সঠিক ও ভুল কোডের উদাহরণ

● CORRECT DESIGN PATTERN

```
# সঠিক উপায়ে ২D ম্যাট্রিক্স বা লিস্ট তৈরি (যাতে মেমোরি বাগ না হয়)
matrix = [[0 for _ in range(3)] for _ in range(3)]
matrix[0][0] = 5
print(matrix) # আউটপুট: [[5, 0, 0], [0, 0, 0], [0, 0, 0]] – শুধু একটিই বদলেছে।
```

● INCORRECT DESIGN PATTERN

```
# ভুল উপায়ে ২D ম্যাট্রিক্স তৈরি
bad_matrix = [[0] * 3] * 3
bad_matrix[0][0] = 5
print(bad_matrix) # ভুল! আউটপুট: [[5, 0, 0], [5, 0, 0], [5, 0, 0]] – সব বদলে গেছে!
```

অধ্যায় ২: অ্যাডভান্সড কমপ্রিহেনশন (Comprehensions)

১. সহজ বাংলা ব্যাখ্যা

- **ডিকশনারি ও সেট কমপ্রিহেনশন:** আমরা পার্ট ১-এ লিস্ট কমপ্রিহেনশন শিখেছিলাম, যার মাধ্যমে কয়েক লাইনের `for` লুপকে এক লাইনে আনা যায়। পাইথনে আপনি একইভাবে এক লাইনে ডিকশনারিও তৈরি করতে পারেন (Dict Comprehension) এবং সেটও তৈরি করতে পারেন (Set Comprehension)।
- **নেস্টেড লিস্ট কমপ্রিহেনশন (Nested List Comprehension):** এটি মূলত লুপের ভেতর লুপ (Nested Loop) চালিয়ে এক লাইনে জটিল টু-ডাইমেনশনাল (2D) ডাটা বা ম্যাট্রিক্স তৈরি করার একটি অত্যন্ত প্রফেশনাল ও পাইথনিক স্টাইল।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
# ১. Dict Comprehension (সংখ্যা এবং তার স্কয়ারের ডিকশনারি)
square_dict = {x: x**2 for x in range(1, 5)}
print(square_dict) # আউটপুট: {1: 1, 2: 4, 3: 9, 4: 16}

# ২. Set Comprehension (ডুপ্লিকেট বাদ দিয়ে ইউনিক সেটের এক লাইনের কোড)
nums = [1, 2, 2, 3, 4, 4]
unique_squares = {x**2 for x in nums}
print(unique_squares) # আউটপুট: {1, 4, 9, 16} (সেটে ডুপ্লিকেট থাকে না)

# ৩. Nested List Comprehension (১ থেকে ৯ পর্যন্ত দিয়ে একটি ৩x৩ ম্যাট্রিক্স তৈরি)
matrix = [(i*3 + j + 1) for j in range(3)] for i in range(3)]
print(matrix) # আউটপুট: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

৩. সঠিক ও ভুল কোডের উদাহরণ

CORRECT DESIGN PATTERN

```
names = ["Asif", "Riya"]
# এক লাইনে নাম ও নামের অক্ষরের সংখ্যার ডিকশনারি তৈরি
name_lengths = {name: len(name) for name in names}
```

● INCORRECT DESIGN PATTERN

```
# ভুল! ডিকশনারি কমপ্রিহেনশনে শুধু একটি ভ্যালু দিলে সেটি ডিকশনারি না হয়ে 'Set' হয়ে যাবে।  
bad_dict = {x for x in range(3)} # এটি আসলে একটি Set তৈরি করল, Dictionary নয়
```

অধ্যায় ৩: পাইথনের স্পেশাল ফাংশন ও টুলস (Special Functions)

১. সহজ বাংলা ব্যাখ্যা

- **Lambda Functions:** এটি হলো এক লাইনের একটি নামহীন বা বেনামী ফাংশন (Anonymous Function)। বড় বড় প্রজেক্টে যেখানে খুব ছোট এবং এককালীন একটি লজিক বা ক্যালকুলেশন দরকার হয়, সেখানে পুরো `def` দিয়ে ফাংশন না লিখে এক লাইনে ল্যাম্বডা ব্যবহার করা হয়।
- **Map, Filter, Reduce:**
 - `map()` : একটি লিস্টের প্রতিটা উপাদানের ওপর কোনো একটি ফাংশন বা লজিক একবারে অ্যাপ্লাই করার জন্য লুপ ছাড়া দ্রুত কাজ করে।
 - `filter()` : একটি লিস্ট থেকে নির্দিষ্ট শর্ত পূরণ করে এমন উপাদানগুলোকে ছেঁকে বা ফিল্টার করে বের করে আনে।
 - `reduce()` : (এটি `functools` মডিউল থেকে আসে) পুরো লিস্টের সব উপাদানকে ক্রমান্বয়ে ক্যালকুলেট করে একটিমাত্র সিঙ্গেল ফলাফলে নামিয়ে আনে (যেমন: সব সংখ্যার গুণফল)।
- **Zip & Enumerate:**
 - `enumerate()` : লুপ চালানোর সময় উপাদানটির পাশাপাশি সেটির ইনডেক্স নাম্বার (0, 1, 2...) কত, তাও ট্র্যাক রাখার সবচেয়ে পরিচ্ছন্ন উপায়।
 - `zip()` : দুটি আলাদা আলাদা লিস্টকে জিপারের মতো কামড়ে ধরে একসাথে সমান্তরালভাবে লুপ করার জন্য ব্যবহার করা হয়।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]

# Lambda এবং Map-Filter এর মেকানিক্স
doubled = list(map(lambda x: x * 2, numbers)) # সব সংখ্যা দ্বিগুণ হবে
evens = list(filter(lambda x: x % 2 == 0, numbers)) # শুধু জোড় সংখ্যা ফিল্টার হবে
total_product = reduce(lambda x, y: x * y, numbers) # সব সংখ্যার গুণফল (১*২*৩*৪*৫)

# Enumerate এবং Zip মেকানিক্স
names = ["Anik", "Sumi"]
scores = [85, 92]
```

```
for index, (name, score) in enumerate(zip(names, scores)):
    print(f"রোল {index+1}: {name} পেয়েছে {score}")
```

৩. সঠিক ও ভুল কোডের উদাহরণ

● CORRECT DESIGN PATTERN

```
# ম্যাপ বা ফিল্টারের আউটপুট সরাসরি প্রিন্ট করলে অবজেক্ট দেখায়, তাই একে list()-এ কনভার্ট করতে হয়।
res = list(map(lambda x: x+1, [1, 2]))
```

● INCORRECT DESIGN PATTERN

```
# ভুল! map() বা filter() সরাসরি প্রিন্ট করলে ডাটা দেখা যায় না।
my_map = map(lambda x: x*2, [1, 2])
print(my_map) # স্ক্রিনে দেখাবে: <map object at 0x...> (Data দেখতে পাবেন না).
```

অধ্যায় ৪: মেমোরি অপ্টিমাইজেশন (Iterators & Generators)

১. সহজ বাংলা ব্যাখ্যা

- **জেনারেটর (Generators) এবং yield:** মনে করুন আপনার কম্পিউটারে একটি বিশাল টেক্সট ফাইল আছে যার সাইজ ১০ গিগাবাইট (10 GB), কিন্তু আপনার কম্পিউটারের র‍্যাম (RAM) মাত্র ৪ গিগাবাইট। আপনি যদি সাধারণ লিস্ট দিয়ে পুরো ফাইল একবারে পড়তে যান, তবে আপনার কম্পিউটার সাথে সাথে ক্র্যাশ করবে। এই সমস্যার একমাত্র সমাধান হলো জেনারেটর।
- জেনারেটরে `return` এর বদলে `yield` কিওয়ার্ড ব্যবহার করা হয়। এটি পুরো ডাটা একবারে মেমোরিতে লোড না করে, যখনই লুপের মাধ্যমে ১টি ডাটা চাওয়া হয়, তখন মেমোরিতে কেবল ১টি ডাটা তৈরি করে ফেরত দেয় এবং পরের ডাতার জন্য চুপচাপ অপেক্ষা করে। একে বলে **Lazy Evaluation**।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
# একটি জেনারেটর ফাংশন যা কোটি কোটি সংখ্যা মেমোরি ছাড়াই জেনারেট করতে পারে
def infinite_sequence(limit):
    num = 1
    while num <= limit:
        yield num # return এর বদলে yield ব্যবহার। এটি স্টেট ধরে রাখে।
        num += 1

# জেনারেটর অবজেক্ট তৈরি (এটি মেমোরিতে ১টি সংখ্যাও এখনো জমা করেনি)
gen = infinite_sequence(1000000)

# প্রয়োজন অনুযায়ী ১টি ১টি করে ডাটা বের করা
print(next(gen)) # আউটপুট: 1
print(next(gen)) # আউটপুট: 2
# বাকি ৯৯৯,৯৯৮ টি সংখ্যা মেমোরিতে জায়গা নষ্ট করছে না!
```

৩. সঠিক ও ভুল কোডের উদাহরণ

CORRECT DESIGN PATTERN

```
def my_gen():
    yield "Step 1"
    yield "Step 2"
```

```
for step in my_gen():  
    print(step)
```

● INCORRECT DESIGN PATTERN

```
def count_up(max_num):  
    for i in range(max_num):  
        return i # ভুল! প্রথম লুপেই 'return' এর কারণে ফাংশন পার্মানেন্টলি বন্ধ হয়ে যাবে।
```

অধ্যায় ৫: কোড মডিফিকেশন (Decorators)

১. সহজ বাংলা ব্যাখ্যা

- **ডেকোরেটর (Decorators):** মনে করুন আপনার একটি সাধারণ ফাংশন আছে যা একটি মেসেজ প্রিন্ট করে। এখন আপনি চান মূল ফাংশনের কোড একটুও পরিবর্তন না করে, সেটির কাজের আগে বা পরে অতিরিক্ত কিছু লজিক জুড়ে দিতে (যেমন: ইউজার লগইন করা আছে কিনা চেক করা, অথবা ফাংশনটি রান হতে কত মিলি-সেকেন্ড সময় লেগেছে তা ট্র্যাক করা)। এই জাদুকরী কাজের টুলের নামই হলো ডেকোরেটর। পাইথনে ফাংশনের উপরে `@ডেকোরেটরের_নাম` লিখে এটি ব্যবহার করা হয়।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
import time

# এটি আমাদের ডেকোরেটর ফাংশন যা অন্য একটি ফাংশনকে র্যাপ (Wrap) করবে
def timer_decorator(original_function):
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = original_function(*args, **kwargs) # আসল ফাংশন রান হলো
        end_time = time.time()
        print(f"[@] {original_function.__name__} ফাংশনটি রান হতে সময় নিয়েছে: {end_time - start_time}")
        return result
    return wrapper

# ডেকোরেটর ব্যবহার করার নিয়ম
@timer_decorator
def complex_calculation():
    time.sleep(1) # ১ সেকেন্ডের জন্য কোড থামিয়ে রাখা হলো
    print("হিসাব নিকাশ সম্পন্ন!")

complex_calculation()
```

৩. সঠিক ও ভুল কোডের উদাহরণ

CORRECT DESIGN PATTERN

```
def my_decorator(func):
    def wrapper(*args, **kwargs): # *args এবং **kwargs দেওয়া সেফ
```

```
print("Before")
return func(*args, **kwargs)
return wrapper
```

● INCORRECT DESIGN PATTERN

```
def bad_decorator(func):
    def wrapper(): # ভুল! যদি আসল ফাংশনে কোনো প্যারামিটার থাকে, তবে এটি ক্র্যাশ করবে।
        print("Start")
        func()
    return wrapper
```

অধ্যায় ৬: ডান্ডার মেথডস (Dunder / Magic Methods)

১. সহজ বাংলা ব্যাখ্যা

- **ডান্ডার মেথডস (Double Underscore):** পাইথনে যে মেথডগুলোর শুরুতে এবং শেষে দুটি করে আন্ডারস্কোর থাকে (যেমন: `__init__`, `__str__`), সেগুলোকে ডান্ডার বা ম্যাজিক মেথড বলে। এগুলো পাইথনের বিল্ট-ইন আচরণের সাথে আমাদের কাস্টম অবজেক্টকে কানেক্ট করে।
- যেমন, আপনি দুটি সাধারণ নাম্বার যোগ করতে `+` ব্যবহার করেন। কিন্তু আপনি যদি নিজের তৈরি করা দুটি ক্লাসের অবজেক্টকে সরাসরি `+` চিহ্ন দিয়ে যোগ করতে চান, তবে পাইথনের ম্যাজিক মেথড `__add__` ওভাররাইড করতে হবে। আবার একটি অবজেক্টকে `print()` করলে যেন হিজিবিজি মেমোরি অ্যাড্রেস না দেখিয়ে সুন্দর লেখা দেখায়, তার জন্য `__str__` ব্যবহার করতে হয়।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    # অবজেক্ট প্রিন্ট করার আচরণ বদলে দেওয়া
    def __str__(self):
        return f"{self.name} - মূল্য: {self.price} টাকা"

    # দুটি অবজেক্ট সরাসরি যোগ (+) করার আচরণ নির্ধারণ
    def __add__(self, other):
        return self.price + other.price

p1 = Product("বই", 250)
p2 = Product("কলম", 50)

print(p1)          # আউটপুট: বই - মূল্য: 250 টাকা (হিজিবিজি অবজেক্ট আইডি দেখাবে না)
print(p1 + p2)     # আউটপুট: 300 (সরাসরি অবজেক্ট যোগ হয়ে গেল ম্যাজিক মেথডের কারণে!)
```

৩. সঠিক ও ভুল কোডের উদাহরণ

● CORRECT DESIGN PATTERN

```
class Book:
    def __str__(self):
        return "Python Guide" # সর্বদা String রিটার্ন করতে হবে
```

● INCORRECT DESIGN PATTERN

```
class BadClass:
    def __str__(self):
        return 100 # ভুল! __str__ মেথড থেকে ভুলেও String ছাড়া অন্য কিছু রিটার্ন করা যাবে না (TypeError আসবে)
```

অধ্যায় ৭: রেগুলার এক্সপ্রেশন (Regex)

১. সহজ বাংলা ব্যাখ্যা

- **রেগুলার এক্সপ্রেশন (Regex):** মনে করুন আপনার কাছে বিশাল একটি টেক্সট ডকুমেন্ট আছে এবং সেখান থেকে আপনাকে সমস্ত সঠিক ইমেইল অ্যাড্রেস, ফোন নাম্বার অথবা নির্দিষ্ট প্যাটার্নের পাসওয়ার্ড খুঁজে বের করতে হবে। সাধারণ `if-else` বা স্ট্রিং মেথড দিয়ে এটি করা অসম্ভব কঠিন। এই জটিল সার্চিং ও প্যাটার্ন ম্যাচিং নিখুঁতভাবে করার ইন্ডাস্ট্রিয়াল টুলের নাম হলো রেগুলার এক্সপ্রেশন বা সংক্ষেপে **Regex**। পাইথনে এর জন্য বিল্ট-ইন `re` মডিউল ব্যবহার করা হয়।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
import re

text = "আমার ইমেইল হলো anik123@gmail.com এবং অফিসের মেইল হলো work_info@yahoo.com"

# ইমেইলের একটি স্ট্যান্ডার্ড Regex প্যাটার্ন
email_pattern = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'

# টেক্সটের ভেতর থেকে সব ম্যাচ খুঁজে বের করা
emails = re.findall(email_pattern, text)
print(emails) # আউটপুট: ['anik123@gmail.com', 'work_info@yahoo.com']
```

৩. সঠিক ও ভুল কোডের উদাহরণ

CORRECT DESIGN PATTERN

```
import re
# র-স্ট্রিং (r"...") ব্যবহার করা ভালো যেন ব্যাকস্ল্যাশ (\) এরর না করে
pattern = r"\d+" # একের অধিক সংখ্যার প্যাটার্ন
match = re.findall(pattern, "দাম হলো ৫০০ টাকা") # ['৫০০']
```

● INCORRECT DESIGN PATTERN

```
# ভুল! re.search() বা re.match() সরাসরি লিস্ট দেয় না, এরা 'Match Object' দেয়।  
result = re.search(r"\d+", "Age: 25")  
print(result) # স্ক্রিনে দেখাবে <re.Match object;...>; ভাটা দেখতে result.group() লাগবে।
```

পার্ট ৪: প্রফেশনাল ওয়ার্কফ্লো, ইকোসিস্টেম ও রিয়েল প্রজেক্টস

অধ্যায় ১: এনভায়রনমেন্ট MANAGEMENT (Virtual Environments & Pip)

১. সহজ বাংলা ব্যাখ্যা

- **ভার্চুয়াল এনভায়রনমেন্টের প্রয়োজনীয়তা:** মনে করুন আপনি কম্পিউটারে দুটি প্রজেক্ট তৈরি করছেন। প্রজেক্ট 'A' তৈরি করা হয়েছে Django ৪.০ ভার্সন দিয়ে, আর প্রজেক্ট 'B' তৈরি করছেন Django ৫.০ ভার্সন দিয়ে। আপনি যদি এগুলো সরাসরি গ্লোবাল কম্পিউটারে ইনস্টল করেন, তবে একটির ভার্সন অন্যটিকে ওভাররাইট করে পুরো প্রজেক্ট ক্র্যাশ করিয়ে দেবে। এই বিপদ থেকে বাঁচতে পাইথনে প্রতিটি প্রজেক্টের জন্য একটি আলাদা, স্বাধীন এবং অদৃশ্য বাক্স বা **Virtual Environment (venv)** তৈরি করা হয়।
- **Pip ও requirements.txt:** pip হলো পাইথনের নিজস্ব প্লে-স্টোর, যা দিয়ে আমরা বাইরের থার্ড-পার্টি লাইব্রেরি (যেমন: Django, FastAPI) ইনস্টল করি। আর আপনার প্রজেক্টটি চালাতে কী কী লাইব্রেরি কত ভার্সনে লাগবে, তার একটি তালিকা তৈরি করে রাখা হয় requirements.txt ফাইলে। এর ফলে অন্য কোনো ডেভেলপার আপনার প্রজেক্টটি নিজের কম্পিউটারে এক লাইনেই সেটআপ করে নিতে পারেন।

২. কোডের সিনট্যাক্স ও মেকানিক্স (টার্মিনাল কমান্ড)

```
# ১. প্রজেক্ট ফোল্ডারে গিয়ে নতুন ভার্চুয়াল এনভায়রনমেন্ট তৈরি করার কমান্ড (উইন্ডোজ/ম্যাক)
python -m venv myenv

# ২. এনভায়রনমেন্টটি একটিভেট (Activate) করার কমান্ড:
# উইন্ডোজে:
myenv\Scripts ctivate
# ম্যাক/লিনাক্সে:
source myenv/bin/activate

# ৩. কোনো লাইব্রেরি ইনস্টল করা এবং লিস্ট তৈরি করা:
pip install requests
pip freeze > requirements.txt # সব ডিপেন্ডেন্সি ফাইলে সেভ হবে

# ৪. অন্য কম্পিউটারে এই ফাইল দেখে সব ইনস্টল করার কমান্ড:
pip install -r requirements.txt
```

৩. সঠিক ও ভুল কোডের উদাহরণ

- **সহীহ প্র্যাকটিস:** প্রজেক্ট শুরু করার আগেই টার্মিনালে চেক করা যে লাইনের শুরুতে `(myenv)` বা আপনার এনভায়রনমেন্টের নাম দেখাচ্ছে কিনা। নাম দেখালে বুঝবেন আপনি সুরক্ষিত বাক্সের ভেতরে আছেন।
- **ভুল প্র্যাকটিস:** এনভায়রনমেন্ট একটিভেট না করেই সরাসরি `pip install` চালানো। এতে আপনার মেইন কম্পিউটার আবর্জনায় ভরে যাবে এবং পরবর্তীতে প্রজেক্ট সার্ভারে আপলোড করার সময় ভার্সন ম্যাচ করবে না।

৪. প্র্যাকটিস প্রবলেম (নিজে চেষ্টা করুন)

1. আপনার কম্পিউটারে `project_test` নামে একটি নতুন ফোল্ডার তৈরি করুন। টার্মিনাল বা সিএমডি (CMD) ব্যবহার করে সেটির ভেতরে একটি ভার্চুয়াল এনভায়রনমেন্ট তৈরি ও একটিভেট করুন।
2. এনভায়রনমেন্ট একটিভেট থাকা অবস্থায় `pip install fastapi` কমান্ড দিয়ে লাইব্রেরিটি ইনস্টল করুন এবং `requirements.txt` ফাইলটি তৈরি করে দেখুন ভেতরে কী লেখা এসেছে।

অধ্যায় ২: প্রফেশনাল প্রজেক্ট আর্কিটেকচার (Project Architecture)

১. সহজ বাংলা ব্যাখ্যা

- **ফোল্ডার স্ট্রাকচার:** প্রো-ডেভেলপাররা কখনোই একটি মাত্র ফাইলের ভেতর হাজার হাজার লাইনের কোড হিজিবিজি করে রাখেন না। তারা কোডকে কাজ অনুযায়ী আলাদা আলাদা ফোল্ডারে ভাগ করেন।
- **__init__.py এর ব্যবহার:** কোনো ফোল্ডারের ভেতরে `__init__.py` নামের একটি ফাঁকা ফাইল রেখে দিলে পাইথন বুঝে নেয় যে এই ফোল্ডারটি কোনো সাধারণ ডিরেক্টরি নয়, এটি একটি **Package** বা মডিউলের কালেকশন। এর ফলে আমরা এক ফোল্ডারের কোড অন্য ফোল্ডারে খুব সহজে `import` করে ব্যবহার করতে পারি।

২. প্রফেশনাল ফোল্ডার লেআউট স্ট্রাকচার

```
my_awesome_project/
|
├── myenv/                # ভার্চুয়াল এনভায়রনমেন্ট (এটি গিটহাবে পুশ করা হয় না)
├── src/                  # সোর্স কোডের মেইন ফোল্ডার
|   ├── __init__.py      # ফোল্ডারটিকে প্যাকেজ বানানোর জন্য ফাঁকা ফাইল
|   ├── database.py      # ডাটাবেজ কানেকশনের কোড
|   └── utils.py         # বিভিন্ন সাহায্যকারী ফাংশন
|
├── tests/               # সমস্ত টেস্ট কেসের ফোল্ডার
|   └── test_main.py
|
├── .gitignore           # কোন কোন ফাইল গিটে আপলোড হবে না তার লিস্ট
├── README.md            # প্রজেক্টের বর্ণনা ও চালানোর নিয়মাবলী
└── requirements.txt     # লাইব্রেরির তালিকা
```

অধ্যায় ৩: ক্লিন কোড ও প্রফেশনাল অভ্যাস (Clean Code)

১. সহজ বাংলা ব্যাখ্যা

- **DRY নীতি (Don't Repeat Yourself):** যদি দেখেন আপনার প্রজেক্টের ৩টি আলাদা জায়গায় আপনি একই লজিক কপি-পেস্ট করেছেন, তবে বুঝবেন আপনার কোড স্ট্যান্ডার্ড হয়নি। নিয়ম হলো, সেই কমন কোডটিকে একটি আলাদা ফাংশন বানিয়ে ফেলুন এবং ৩ জায়গা থেকে তাকে কল করুন।
- **Type Hinting:** পাইথনে ভ্যারিয়েবলের কোনো নির্দিষ্ট টাইপ বলে দিতে হয়। কিন্তু বড় প্রজেক্টে কোন ফাংশনে কী ডেটা ঢুকছে আর কী বের হচ্ছে তা বুঝতে অসুবিধা হয়। তাই আধুনিক পাইথনে টাইপ হিন্টিং ব্যবহার করা হয়, যা কোডের রিডাবিলিটি (পড়ার যোগ্যতা) ১০০ গুণ বাড়িয়ে দেয়।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
# টাইপ হিন্টিং সহ ক্লিন কোড:  
# এখানে স্পষ্ট বলা আছে 'name' হবে string, 'age' হবে int এবং ফাংশনটি ফেরত দেবে string (-> str)  
def generate_greeting(name: str, age: int) -> str:  
    return f"Hello {name}, you are {age} years old."  
  
message: str = generate_greeting("Rakib", 26)  
print(message)
```

অধ্যায় ৪: রিয়েল-ওয়ার্ল্ড প্রজেক্ট ইমপ্লিমেন্টেশন (The Pro Projects)

প্রজেক্ট ১: CLI ভিত্তিক সিকিউর লগইন, সাইনআপ ও পাসওয়ার্ড হ্যাশিং সিস্টেম

```
import json
import hashlib
import os
from typing import Dict, Any

class SecuritySystem:
    def __init__(self, db_filename: str = "users.json") -> None:
        self.db_filename = db_filename
        self.users: Dict[str, str] = self._load_database()

    def _load_database(self) -> Dict[str, str]:
        if not os.path.exists(self.db_filename):
            return {}
        try:
            with open(self.db_filename, "r") as file:
                return json.load(file)
        except json.JSONDecodeError:
            print("[!] ডাটাবেজ ফাইলটি ক্ষতিগ্রস্ত! নতুন ডাটাবেজ ক্রিয়েট হচ্ছে!")
            return {}

    def _save_database(self) -> None:
        with open(self.db_filename, "w") as file:
            json.dump(self.users, file, indent=4)

    def _hash_password(self, password: str) -> str:
        return hashlib.sha256(password.encode()).hexdigest()

    def register(self, username: str, password: str) -> bool:
        clean_username = username.strip().lower()
        if clean_username in self.users:
            print("[X] দুঃখিত, এই ইউজারনেমটি ইতিপূর্বেই ব্যবহার করা হয়েছে!")
            return False

        if len(password) < 6:
            print("[X] পাসওয়ার্ড অবশ্যই সর্বনিম্ন ৬ ডিজিটের হতে হবে!")
            return False
```

```

self.users[clean_username] = self._hash_password(password)
self._save_database()
print(f"[✓] অভিনন্দন {username}, আপনার রেজিস্ট্রেশন সফল হয়েছে!")
return True

def login(self, username: str, password: str) -> bool:
    clean_username = username.strip().lower()
    stored_hash = self.users.get(clean_username)

    if not stored_hash:
        print("[X] ইউজারনেম খুঁজে পাওয়া যায়নি!")
        return False

    if stored_hash == self._hash_password(password):
        print(f"[✓] লগইন সফল! স্বাগতম, {username}!")
        return True
    else:
        print("[X] ভুল পাসওয়ার্ড! আবার চেষ্টা করুন!")
        return False

def main():
    system = SecuritySystem()
    print("=== অ্যাডভান্সড সিকিউরিটি অ্যান্ড অথ সিস্টেম ===")

    while True:
        choice = input("
১. সাইনআপ (Register)
২. লগইন (Login)
৩. বের হয়ে যান (Exit)
অপশন সিলেক্ট করুন: ")

        if choice == "1":
            user = input("নতুন ইউজারনেম দিন: ")
            pwd = input("পাসওয়ার্ড দিন: ")
            system.register(user, pwd)
        elif choice == "2":
            user = input("আপনার ইউজারনেম: ")
            pwd = input("আপনার পাসওয়ার্ড: ")
            system.login(user, pwd)
        elif choice == "3":
            print("প্রোগ্রামটি ব্যবহার করার জন্য ধন্যবাদ। বিদায়!")
            break
        else:
            print("[!] ভুল ইনপুট! দয়া করে ১, ২ বা ৩ চাপুন!")

if __name__ == "__main__":
    main()

```

প্রজেক্ট ২: অবজেক্ট-ওরিয়েন্টেড ডেইলি কস্ট ট্রাকার (Daily Expense Tracker)

```
import json
import os
from datetime import datetime
from typing import List, Dict, Any

class Expense:
    def __init__(self, amount: float, category: str, description: str) -> None:
        self.amount = amount
        self.category = category.strip().title()
        self.description = description
        self.date = datetime.now().strftime("%Y-%m-%d %H:%M")

    def to_dict(self) -> Dict[str, Any]:
        return self.__dict__

class ExpenseTracker:
    def __init__(self, file_path: str = "expenses.json") -> None:
        self.file_path = file_path
        self.expenses: List[Dict[str, Any]] = self._load_expenses()

    def _load_expenses(self) -> List[Dict[str, Any]]:
        if not os.path.exists(self.file_path):
            return []
        try:
            with open(self.file_path, "r") as file:
                return json.load(file)
        except IOError:
            return []

    def _save_expenses(self) -> None:
        with open(self.file_path, "w") as file:
            json.dump(self.expenses, file, indent=4)

    def add_expense(self, amount: float, category: str, description: str) -> None:
        if amount <= 0:
            print("[X] খরচের পরিমাণ অবশ্যই ০ এর বেশি হতে হবে!")
            return

        new_expense = Expense(amount, category, description)
        self.expenses.append(new_expense.to_dict())
        self._save_expenses()
        print(f"[✓] {amount} টাকা '{category}' ক্যাটাগরিতে সাকসেসফুলি যোগ হয়েছে!")
```

```

def view_all_expenses(self) -> None:
    if not self.expenses:
        print("[!] এখনো কোনো খরচের ডাটা এন্ট্রি করা হয়নি।")
        return

    print("
    --- আপনার সমস্ত খরচের তালিকা ---")
    print(f"{'তারিখ':<18} | {'ক্যাটাগরি':<12} | {'পরিমাণ (টাকা)':<14} | {'বিবরণ'}")
    print("-" * 65)
    for exp in self.expenses:
        print(f"{exp['date']:<18} | {exp['category']:<12} | {exp['amount']:<14.2f} | {exp['desc']:<18}")

    total = sum(map(lambda x: x['amount'], self.expenses))
    print("-" * 65)
    print(f"মোট সর্বমোট খরচ: {total:.2f} টাকা।")

def main():
    tracker = ExpenseTracker()
    print("=== ওওপি ভিত্তিক কাস্টম ডেইলি এক্সপেন্স ট্র্যাকার ===")

    while True:
        menu = input("
        ১. নতুন খরচ যোগ করুন (Add Expense)
        ২. সমস্ত খরচ ও টোটাল দেখুন (View Expenses)
        ৩. বন্ধ করুন (Exit)
        অপশন দিন: ")

        if menu == "1":
            try:
                taka = float(input("কত টাকা খরচ হয়েছে?: "))
                cat = input("ক্যাটাগরি (যেমন: Food, Rent, Travel): ")
                desc = input("খরচের সংক্ষিপ্ত বিবরণ: ")
                tracker.add_expense(taka, cat, desc)
            except ValueError:
                print("[X] ভুল ইনপুট! টাকার ঘরে দয়া করে সঠিক সংখ্যা লিখুন।")
        elif menu == "2":
            tracker.view_all_expenses()
        elif menu == "3":
            print("[✓] আপনার হিসাব সুরক্ষিত আছে। ধন্যবাদ!")
            break
        else:
            print("[!] দয়া করে সঠিক অপশন (১, ২ বা ৩) সিলেক্ট করুন।")

if __name__ == "__main__":
    main()

```



পার্ট ৫: দি আলটিমেট প্রো প্লাস (Pro+ Concepts)

অধ্যায় ১: কনকারেন্সি ও প্যারালেলিজম (Asyncio, Threading & Multiprocessing)

১. সহজ বাংলা ব্যাখ্যা

- **প্রবলেম মেকানিক্স:** আমরা এতক্ষণ যে কোড লিখেছি, তা ছিল 'সিঙ্ক্রোনাস' (Synchronous)। অর্থাৎ, লাইন ১-এর কাজ শেষ না হলে লাইন ২-এর কাজ শুরু হতো না। কিন্তু ধরুন, আপনাকে ইন্টারনেট থেকে ১০০টি ফাইল ডাউনলোড করতে হবে। একটার পর একটা করতে গেলে অনেক সময় লাগবে।
- **মাল্টিথ্রেডিং (Multi-threading):** এটি এক ধরনের ভেক্টিবাজি। যখন একটি টাস্ক অলস বসে থাকে (যেমন ইন্টারনেটের জন্য ওয়েট করছে), তখন প্রসেসর অন্য আরেকটি টাস্কের কাজ শুরু করে দেয়। একে বলে I/O Bound Task অপটিমাইজেশন।
- **অ্যাসিনক্রোনাস (Asyncio):** আধুনিক পাইথনে থ্রেড না বাড়িয়ে এক লাইনের ভেতর `async` এবং `await` কিওয়ার্ড ব্যবহার করে খুব দ্রুত হাজার হাজার রিকোয়েস্ট একসাথে হ্যান্ডেল করা যায়। একে বলে Non-blocking কোড।
- **মাল্টিপ্রসেসিং (Multiprocessing):** পাইথনে **GIL (Global Interpreter Lock)** নামের একটি লক থাকার কারণে একটি কোর একসাথে কেবল একটি থ্রেডই চালাতে পারে। তাই যদি খুব জটিল গাণিতিক হিসাব বা ইমেজ প্রসেসিংয়ের কাজ থাকে (CPU Bound Task), তবে কম্পিউটারের মাল্টিপল প্রসেসর বা কোরকে একসাথে ব্যবহার করার জন্য মাল্টিপ্রসেসিং ব্যবহার করা হয়।

২. কোডের সিনট্যাক্স ও মেকানিক্স (Asyncio)

```
import asyncio

async def fetch_data(id: int, delay: int):
    print(f"[*] ডাটা {id} ডাউনলোড শুরু হলো...")
    await asyncio.sleep(delay)
    print(f"[✓] ডাটা {id} ডাউনলোড শেষ!")
    return {"id": id, "data": "success"}

async def main():
    task1 = asyncio.create_task(fetch_data(1, 2))
```

```
task2 = asyncio.create_task(fetch_data(2, 3))
task3 = asyncio.create_task(fetch_data(3, 1))

await asyncio.gather(task1, task2, task3)

asyncio.run(main())
```

৩. সঠিক ও ভুল কোডের উদাহরণ

● CORRECT DESIGN PATTERN

```
async def download():
    await asyncio.sleep(1) # সঠিক: অ্যাসিনক্রোনাস স্লিপ
```

● INCORRECT DESIGN PATTERN

```
async def download():
    time.sleep(1) # ভুল! অ্যাসিনক্রোনাস ফাংশনের ভেতর নরমাল time.sleep দিলে পুরো প্রজেক্ট জ্যাম বা ব্লক হয়ে যাবে।
```

অধ্যায় ২: পাইথন ডেটা স্ট্রাকচার ডিপ ডাইভ (Collections & Itertools)

১. সহজ বাংলা ব্যাখ্যা

- **বিল্ট-ইন অ্যাডভান্সড কালেকশনস:** আমরা লিস্ট, ডিকশনারি তো জানিই। কিন্তু পাইথনের `collections` এবং `itertools` মডিউলে এমন কিছু প্রো-লেভেল ডেটা স্ট্রাকচার আছে যা কোডের পারফরম্যান্স এবং স্পিড বহুগুণ বাড়িয়ে দেয়।
 - `namedtuple` : অবজেক্টের মতো কাজ করে কিন্তু মেমোরি খরচ একদম কমায়।
 - `defaultdict` : ডিকশনারিতে কোনো Key না থাকলে ত্র্যাশ করার বদলে স্বয়ংক্রিয়ভাবে একটি ডিফল্ট ভ্যালু (যেমন `0` বা `[]`) সেট করে দেয়।
 - `Counter` : একটি বিশাল লিস্টে কোন উপাদান কতবার আছে, তা লুপ ছাড়াই এক সেকেন্ডে গুনে দেয়।

২. কোডের সিনট্যাক্স ও মেকানিক্স

```
from collections import Counter, defaultdict

votes = ["apple", "banana", "apple", "orange", "banana", "apple"]
vote_count = Counter(votes)
print(vote_count)
print(vote_count.most_common(1))

dict_with_default = defaultdict(int)
dict_with_default["missing_key"] += 1
print(dict_with_default["missing_key"])
```

অধ্যায় ৩: গিট ও গিটহাব প্রফেশনাল ওয়ার্কফ্লো (Git & GitHub)

১. সহজ বাংলা ব্যাখ্যা

- **ভার্সন কন্ট্রোল:** আপনি আপনার কোডে আজ একটি নতুন ফিচার যোগ করলেন, কিন্তু কোডটি ভেঙে গেল। এখন আপনি চান গতকাল আপনার কোড যেমন ঠিকঠাক ছিল, সেখানে ফেরত যেতে। এই ব্যাকআপ এবং টাইম-ট্রাভেল করার টুলের নামই হলো **Git**। আর আপনার এই কোড অনলাইনে নিরাপদে জমা রাখার ক্লাউড স্টোরেজের নাম হলো **GitHub**।

২. প্রধান কমান্ড মেকানিক্স (টার্মিনাল)

```
git init                # আপনার প্রজেক্ট ফোল্ডারে গিট চালু করার কমান্ড
git add .               # সমস্ত ফাইলের বর্তমান অবস্থার একটি স্ন্যাপশট নেওয়া
git commit -m "ফ্লিন কোড" # স্ন্যাপশটটি একটি মেসেজসহ ইতিহাসে সেভ করা
git branch -M main      # মেইন ব্রাঞ্চ সিলেক্ট করা
git remote add origin URL # গিটহাবের সাথে আপনার কম্পিউটার লিঙ্ক করা
git push -u origin main  # আপনার কোড গিটহাবে আপলোড করে দেওয়া
```

অধ্যায় ৪: আপনার পরবর্তী রিয়েল-ওয়ার্ল্ড ডিরেকশন (The Frameworks)

আপনি পাইথনের Core এবং Advanced পার্টে এখন ১০০% মাস্টার। এখান থেকে একজন রিয়েল-ওয়ার্ল্ড সফটওয়্যার ইঞ্জিনিয়ার হিসেবে ক্যারিয়ার শুরু করতে আপনাকে যেকোনো ১টি ডিরেকশন বা ফ্রেমওয়ার্ক বেছে নিতে হবে:

ডিরেকশন / ট্র্যাক	প্রধান ফ্রেমওয়ার্ক (Frameworks)	এই ট্র্যাকে আপনার কাজ কী হবে?
১. ওয়েব ডেভেলপমেন্ট (Web Backend)	Django অথবা FastAPI	বিভিন্ন নামী ওয়েবসাইটের ব্যাকএন্ড লজিক, ডাটাবেজ আর্কিটেকচার এবং API তৈরি করা।
২. ডাটা সায়েন্স ও এআই (Data & AI)	NumPy, Pandas, Scikit-Learn	ডাটা অ্যানালাইসিস, গ্রাফ তৈরি এবং চ্যাটজিপিটির মতো কাস্টম আর্টিফিশিয়াল ইন্টেলিজেন্স বা মডেল তৈরি করা।
৩. স্ক্র্যাপিং ও অটোমেশন	BeautifulSoup, Selenium	বোরিং ও রিপোর্টেটিভ কাজ অটোমেট করা এবং ইন্টারনেট থেকে ডাটা স্ক্র্যাপ করে আনা।